

Python Fluency

From English requirement to working code

One concept per page. Every page has the same four parts:

KNOW THIS - the syntax to memorize

HOW A QUESTION IS SOLVED - requirement -> inputs -> output -> steps -> Python

TIPS AND TRAPS - what goes wrong and how to avoid it

NOW YOU TRY - four questions, micro to real, no solutions in this book

Road to LC King

How to use this book

Understanding a page is worth nothing. Only code you write yourself counts.

THE LOOP FOR EVERY PAGE

1. Read KNOW THIS once. Then close the book and write the syntax from memory in a blank file.
2. Read the worked requirement. COVER the solution. Do inputs, output and steps yourself first.
3. Write the code. Run it. Compare with the book only AFTER your own attempt.
4. Read TIPS AND TRAPS. These are your most likely bugs.
5. Do the four NOW YOU TRY questions from a BLANK file. Micro first, real requirement last.
6. Stuck? Do not look at the answer. Ask: what do I receive, what do I return, what would I do by hand?
7. Log each mistake in ERROR_LOG.md. Review it before the next session.

MASTERY LEVELS (aim for 7 on every concept)

1. Recognize it
2. Explain it
3. Write it with a hint
4. Write it independently
5. Use it in an unfamiliar problem
6. Debug it
7. Use it under time pressure

DIFFICULTY LADDER

MICRO: one operation. COMBO: 2-3 operations. SMALL PROGRAM: several concepts. REAL REQUIREMENT: English only, you pick the tools. INTERVIEW: unfamiliar, no hints.

START HERE

The 10-step method

Run this on EVERY problem. Never start with code.

1	Restate	Say it in your own words in one sentence.
2	Inputs	What do I receive? What types?
3	Output	What do I return or print? What type?
4	Rules	Write every rule as a sentence. Underline boundary words (at least, over, up to).
5	By hand	Do one example manually with real numbers.
6	Tools	variable? if/elif? loop? list? dict? set? function? (next page)
7	Pseudocode	4-8 plain-English steps.
8	Code	Turn each pseudocode line into Python.
9	Test	Normal, boundary, empty/zero, one item, invalid.
10	Debug	Print values. The bug is the first line where reality differs from your prediction.

IF YOU FREEZE

Ask yourself, in this order: What information do we receive? What do we need to return? What would I do manually with a small example? Then convert each manual step into one line of code. Say out loud: "Let me do an example by hand first."

Which tool does the English point to?

Read the requirement and spot these phrases.

English says	Think	Python
if / otherwise / when / unless	decision	<code>if / elif / else</code>
for each / every / all	repeat over items	<code>for x in items:</code>
until / as long as / keep asking	repeat with a stop	<code>while cond:</code>
total / sum / add up	accumulator	<code>total += x</code>
count how many	counter	<code>count += 1</code>
largest / smallest / best	best so far	<code>if x > best: best = x</code>
list of / in order / many values	list	<code>nums = []</code>
price of X / how many of each	dictionary	<code>d[key]</code>
duplicate / unique / seen before	set (or dict)	<code>seen = set()</code>
calculate / reusable / returns	function	<code>def f(): return</code>
at least / at most / over / under	comparison + boundary	<code>>= <= > <</code>
and / or / both / either	boolean logic	<code>and or not</code>
first N / last N / every other	slicing	<code>a[:N] a[-N:] a[::2]</code>
cap / never more than	<code>min()</code>	<code>min(total, 20)</code>
minimum charge / at least \$5	<code>max()</code>	<code>max(total, 5)</code>
each additional / after the first	subtract the first	<code>base + rate * (n - 1)</code>

Variables and types

A variable is a named box that holds a value. The type of the value decides what you can do with it.

KNOW THIS

```
price = 4.5           # float  (decimal)
count = 3             # int    (whole number)
name = "Mug"          # str    (text, needs quotes)
in_stock = True       # bool   (True / False, capital letter)
total = price * count # right side is calculated, then stored
print(type(price))    # <class 'float'>
print(name, total)    # Mug 13.5
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"A mug costs \$4.50. A customer buys 3. Show the total cost."

Inputs: price = 4.5, quantity = 3

Output: the total cost (a number)

Steps by hand (pseudocode)

1. Store the price in a variable.
2. Store the quantity in a variable.
3. Multiply them and store the result.
4. Print the result.

Python

```
price = 4.5
quantity = 3
total_cost = price * quantity
print(total_cost)
```

TIPS AND TRAPS

- Name variables after what they hold: `total_cost`, not `x`.
- `=` stores a value. `==` compares two values. Never mix them up.
- `"5"` (text) and `5` (number) are different things. `"5" + 1` is an error.
- Convert when needed: `int("7")`, `float("2.5")`, `str(10)`.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Make variables for your name and age. Print both on one line.
2. **COMBO** Store a temperature in Celsius. Compute and print Fahrenheit: `c * 9 / 5 + 32`.
3. **REAL** A phone costs 300 and tax is 8%. Print the price, the tax amount, and the final price.
4. **COMBO** Swap the values of `a = 5` and `b = 9` without losing either value.

Input and output

`input()` asks the user for text. `print()` shows results. Everything typed in arrives as a string, so convert it before doing math.

KNOW THIS

```
name = input("Name: ")          # always a str
age = int(input("Age: "))       # whole number
rate = float(input("Rate: "))   # decimal number

print("Hi", name)               # Hi Sam
print(f"Hi {name}, you are {age}") # f-string: variables inside {}
print(f"Pay: {rate * 8:.2f}")    # :.2f = exactly 2 decimals
print("a", "b", sep="-")        # a-b
print("no newline", end="")     # next print continues on same line
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Ask for a distance in kilometres. Print it in miles (1 km = 0.621 miles), with 2 decimals."

Inputs: km, typed by the user (text -> float)

Output: a line of text with miles to 2 decimals

Steps by hand (pseudocode)

1. Ask for the distance and convert the text to a float.
2. `miles = km * 0.621`
3. Print a sentence with miles formatted to 2 decimals.

Python

```
km = float(input("Distance in km: "))
miles = km * 0.621
print(f"{km} km is {miles:.2f} miles")
```

TIPS AND TRAPS

- `input()` ALWAYS returns text. `"10" + "5"` is `"105"`, not 15.
- Forgot `int()`/`float()`? Expect a `TypeError`.
- An f-string needs the `f` BEFORE the opening quote: `f"...{x}..."`.
- Your prompt should tell the user what to type: `"Distance in km: "`.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Ask for a name and print: Hello, <name>!
2. **MICRO** Ask for a word and print it 3 times separated by dashes.
3. **COMBO** Ask for two numbers. Print their sum and their product on separate lines.
4. **REAL** Ask for hours worked and hourly rate. Print the pay with exactly 2 decimals.

Arithmetic

Python does math with operators. Two of them, `//` and `%`, solve most 'split into parts' problems.

KNOW THIS

```
7 + 2    7 - 2    7 * 2    # 9 5 14
7 / 2    # 3.5    (always a decimal)
7 // 2   # 3      (whole-number division)
7 % 2    # 1      (remainder)
2 ** 5   # 32     (power)
total += 5    # same as total = total + 5    (also -= *= /=)
abs(-4) min(3, 9) max(3, 9) round(3.14159, 2) # 4 3 9 3.14
n % 2 == 0    # True when n is even
n % 10       # last digit      n // 10 # all but last digit
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Convert 275 minutes into hours and remaining minutes."

Inputs: total_minutes = 275

Output: hours and minutes, e.g. 4 hours 35 minutes

Steps by hand (pseudocode)

1. Whole hours = total_minutes // 60.
2. Leftover minutes = total_minutes % 60.
3. Print both.

Python

```
total_minutes = 275
hours = total_minutes // 60
minutes = total_minutes % 60
print(f"{hours} hours {minutes} minutes")
```

TIPS AND TRAPS

- `//` and `%` come as a pair: how many full groups, and what is left over.
- Use `/` for averages and prices. Use `//` for counting whole things.
- `2 ** 3` is 8. `^` means something else in Python. Do not use it.
- Percent: 15% of x is `x * 15 / 100` or `x * 0.15`.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Print the last digit of 4527.
2. **MICRO** Print whether 38 is even.
3. **COMBO** Convert 4000 seconds into hours, minutes and seconds.
4. **REAL** A \$86 bill is split between 4 people. Print each share and any leftover cents when working in whole cents.

Comparisons and booleans

A comparison asks a yes/no question and answers with True or False. and / or / not combine answers. Conditions are built from these.

KNOW THIS

```

==  !=  >  <  >=  <=      # comparisons give True or False
score >= 50                # at least 50
score > 50                 # more than 50 (50 itself is False)
5 <= x <= 10              # between 5 and 10, both ends included
a and b                   # True only if BOTH are True
a or b                    # True if EITHER is True
not a                     # flips True <-> False
day in ("Sat", "Sun")     # True if day matches one of these

```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"A student passes if the score is at least 50 and attendance is at least 75%. Print True or False."

Inputs: score, attendance (numbers)

Output: True or False

Steps by hand (pseudocode)

1. Compare score to 50 using at-least.
2. Compare attendance to 75 using at-least.
3. Join the two checks with **and**.
4. Print the result.

Python

```

score = 62
attendance = 70
passed = score >= 50 and attendance >= 75
print(passed)          # False, attendance is too low

```

TIPS AND TRAPS

- Underline the boundary words: "at least" is **>=**, "more than" is **>**, "at most" is **<=**, "under" is **<**.
- Always test the exact boundary value (50, 75). That is where bugs live.
- **x == 5 or 6** is WRONG. Write **x == 5 or x == 6**, or **x in (5, 6)**.
- A comparison result can be stored in a variable, like **passed** above.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Print whether `18 <= 17` is True or False, then work out why before you run it.
2. **MICRO** Is a number divisible by both 3 and 5? Test it with 15, 9, 30.
3. **COMBO** Print True if a temperature is between 15 and 25 inclusive.
4. **REAL** A promo applies if the cart total is over \$40 OR the customer is a member. Print whether it applies for several cases.

if / else

An if statement runs a block only when a condition is True. else runs when it is not. Indentation (4 spaces) marks what belongs inside.

KNOW THIS

```
if temperature > 30:           # condition, then a COLON
    print("Too hot")           # indented = inside the if
else:                           # no condition on else, just a colon
    print("OK")
print("always runs")           # not indented = outside

if is_raining:                 # a bool needs no == True
    print("Take umbrella")
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"A thermostat alarm: if the temperature is above 30, print Too hot, otherwise print OK."

Inputs: temperature (number from the user)

Output: one line: Too hot, or OK

Steps by hand (pseudocode)

1. Read the temperature as a number.
2. Is it greater than 30?
3. If yes, print Too hot.
4. Otherwise, print OK.

Python

```
temperature = float(input("Temperature: "))
if temperature > 30:
    print("Too hot")
else:
    print("OK")
```

TIPS AND TRAPS

- Forgot the `:` after `if / else`? That is a `SyntaxError`.
- Exactly 30 here prints OK, because "above" means `>`. Test the boundary every time.
- Print EXACTLY the text the requirement asks for, including capital letters.
- A common mistake: a function with an if inside is defined but never called. Nothing happens until you call it.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Print Pass if a score is 50 or more, otherwise Fail.
2. **MICRO** Print Even or Odd for a number the user enters.
3. **COMBO** A cinema charges \$8 for under 12, \$12 otherwise. Ask for age and print the price.
4. **REAL** A bank blocks a withdrawal if the amount is more than the balance. Print Approved or Declined. Test with an amount equal to the balance.

elif chains

When there are more than two outcomes, chain conditions with `elif`. Python checks top to bottom and runs **ONLY** the first `True` branch.

KNOW THIS

```
if usage <= 10:
    level = "Low"
elif usage <= 30:           # only reached if usage > 10
    level = "Medium"
elif usage <= 60:
    level = "High"
else:                       # everything else
    level = "Extreme"
print(level)
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"A water bill label: up to 10 units is Low, up to 30 is Medium, up to 60 is High, anything above is Extreme."

Inputs: usage (number)

Output: a label: Low, Medium, High or Extreme

Steps by hand (pseudocode)

1. Check the smallest range first: `usage <= 10`.
2. Then `usage <= 30` (we already know it is above 10).
3. Then `usage <= 60`.
4. Everything else is Extreme (use `else`).

Python

```
usage = float(input("Units used: "))
if usage <= 10:
    label = "Low"
elif usage <= 30:
    label = "Medium"
elif usage <= 60:
    label = "High"
else:
    label = "Extreme"
print(label)
```

TIPS AND TRAPS

- ORDER MATTERS. Go smallest-to-largest with `<=`, or largest-to-smallest with `>=`. Never mix.
- Wrong order example: checking `usage <= 60` first would label 5 as High.
- Use `else` for "everything not covered above" and also to catch bad input.
- Separate `if` statements are ALL checked. `elif` stops at the first match.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Grade from a score: 90+ A, 80+ B, 70+ C, else F.
2. **COMBO** Delivery fee by distance: up to 5 km \$3, up to 15 km \$6, over 15 km \$10.
3. **COMBO** Print Negative, Zero or Positive for a number.
4. **REAL** Phone battery: below 10% Critical, below 30% Low, below 80% Good, otherwise Full. Test 9, 10, 29, 30, 79, 80, 100.

and / or / not and nested ifs

Real rules combine several facts. Join them with and / or / not, or place one if inside another.

KNOW THIS

```
if is_member and hour >= 6 and hour < 22:    # all must be true
    print("Welcome")

if day == "Sat" or day == "Sun":             # one is enough
    price = 12

if not is_banned:
    ...

# nested: second question only matters if the first is true
if has_ticket:
    if age >= 12:
        print("Enter")
    else:
        print("Kids area")
else:
    print("Buy a ticket")
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"A gym admits you if you are a member AND it is open (6:00 up to but not including 22:00). Otherwise print which reason applies."

Inputs: is_member (bool), hour (int 0-23)

Output: Welcome, Not a member, or Closed

Steps by hand (pseudocode)

1. If not a member -> Not a member.
2. Else if the hour is outside 6..21 -> Closed.
3. Otherwise -> Welcome.

Python

```
is_member = True
hour = 22
if not is_member:
    print("Not a member")
elif hour < 6 or hour >= 22:
    print("Closed")
else:
    print("Welcome")
```

TIPS AND TRAPS

- Say the rule out loud as a sentence, then translate word for word.
- "Not between 6 and 22" is `hour < 6 or hour >= 22`. De Morgan: flip the comparisons AND swap and/or.
- Deep nesting is hard to read. Prefer early checks for the failing cases.
- Python does `5 < x < 10` correctly. It is perfectly valid.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Print Discount if age < 12 or age >= 65.
2. **COMBO** A ride is allowed if height >= 120 and the person has a ticket. Print the reason when denied.
3. **COMBO** Login: correct username AND correct password. Different messages for each failure.
4. **REAL** Shipping is free if the order is over \$50 OR the customer is a member; otherwise \$7. Express adds \$10 whatever else happens.

Strings

Text is a sequence of characters. You can index it, slice it, search it and loop over it. It never changes in place.

KNOW THIS

```
s = "Hello World"
len(s)                # 11
s[0]   s[-1]          # 'H'   'd'
s[0:5]   s[:5]   s[6:] # 'Hello' 'Hello' 'World'
s[::-1]   # 'dlrow olleH' (reverse)
s.lower() s.upper() s.strip() # return NEW strings
s.split() # ['Hello', 'World']
"-".join(["a", "b"])      # 'a-b'
s.replace("l", "L")   s.count("l")   s.find("w")
"World" in s          # True
s.isdigit() s.isalpha() s.isupper()
for ch in s: ...      # one character at a time
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"A username is valid if it has at least 5 characters and contains only letters and digits."

Inputs: username (str)

Output: True or False

Steps by hand (pseudocode)

1. Check `len(username) >= 5`.
2. Check `username.isalnum()` (letters and digits only).
3. Both must be true, so combine with `and`.

Python

```
username = "sam99"
valid = len(username) >= 5 and username.isalnum()
print(valid)      # True
```

TIPS AND TRAPS

- Strings are immutable. `s.upper()` does nothing unless you save it: `s = s.upper()`.
- Slices exclude the stop: `s[0:5]` is characters 0..4.
- Compare text ignoring case: `a.lower() == b.lower()`.
- `"abc" * 2` is `"abcabc"`, but `"abc" + 2` is a `TypeError`.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Print the first and last character of a word.
2. **MICRO** Print a word reversed.
3. **COMBO** Count the vowels in a sentence.
4. **REAL** Password rule: at least 8 chars, contains a digit, contains an uppercase letter. Print which rules fail.

Functions and return

A function is a named recipe. Parameters are its inputs. `return` hands back its output. Defining does nothing until you call it.

KNOW THIS

```
def calculate_tip(bill, percent):          # parameters = inputs
    tip = bill * percent / 100
    return tip                             # output, function ends here

result = calculate_tip(80, 15)             # CALL it. result is 12.0
print(result)

def greet(name, loud=False):              # default value = optional input
    if loud:
        return name.upper() + "!"
    return "Hi " + name

def min_max(nums):
    return min(nums), max(nums)           # two values back
lo, hi = min_max([4, 9, 1])
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Write `calculate_total(price, quantity, tax_percent)` that returns the final cost including tax, rounded to 2 decimals."

Inputs: price, quantity, tax_percent

Output: a number: the final cost

Steps by hand (pseudocode)

1. subtotal = price * quantity.
2. tax = subtotal * tax_percent / 100.
3. Return round(subtotal + tax, 2).

Python

```
def calculate_total(price, quantity, tax_percent):
    subtotal = price * quantity
    tax = subtotal * tax_percent / 100
    return round(subtotal + tax, 2)

print(calculate_total(4.5, 3, 8))        # 14.58
```

TIPS AND TRAPS

- `return` gives a value back. `print` only shows it. Requirement says return? Use return.
- Forgot `return`? The function gives back `None`.
- A `def` alone runs nothing. You must CALL it.
- Plan the signature first: name, inputs, output. Then write the body.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Write `square(n)` that returns $n * n$.
2. **MICRO** Write `is_even(n)` returning True or False.
3. **COMBO** Write `average(a, b, c)`. Call it with three different sets.
4. **REAL** Write `final_price(price, is_member)`: members get 10% off. Return the price rounded to 2 decimals.

for loops and range

A for loop repeats a block once for each item. `range()` makes a sequence of numbers to loop over.

KNOW THIS

```
for x in [4, 8, 15]:           # each item in a list
    print(x)
for ch in "hey":               # each character
    print(ch)

range(5)                       # 0 1 2 3 4           (stop is NOT included)
range(1, 6)                    # 1 2 3 4 5
range(0, 10, 2)                # 0 2 4 6 8           (step 2)
range(5, 0, -1)                # 5 4 3 2 1           (count down)

for i in range(1, 4):
    print(i)
print("done")                  # outside loop: runs once at the end
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Print a countdown from 5 to 1, then print Liftoff."

Inputs: none (fixed range)

Output: 5 4 3 2 1 then Liftoff, one per line

Steps by hand (pseudocode)

1. Loop `i` from 5 down to 1.
2. Print `i` each time.
3. After the loop, print Liftoff.

Python

```
for i in range(5, 0, -1):
    print(i)
print("Liftoff")
```

TIPS AND TRAPS

- `range(a, b)` stops BEFORE `b`. For 1..10 write `range(1, 11)`.
- Everything indented under the `for` repeats. The first unindented line ends the loop.
- Do not change the list while looping over it.
- To repeat `N` times and ignore the number: `for _ in range(N):`.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Print the numbers 1 to 10.
2. **MICRO** Print only the even numbers from 1 to 20.
3. **COMBO** Print the 7 times table up to 7 x 10, formatted like `7 x 3 = 21`.
4. **REAL** Print a table of Celsius 0, 10, 20 ... 100 next to Fahrenheit.

while loops

A while loop repeats as long as its condition is True. YOU must change something inside so it eventually becomes False.

KNOW THIS

```
count = 0
while count < 3:           # check first, then run block
    print(count)
    count += 1             # without this: infinite loop (Ctrl+C stops it)

while True:                # repeat until I say stop
    answer = input("> ")
    if answer == "quit":
        break              # leave the loop now
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"A savings account starts at \$500 and grows 10% each year. How many years until it exceeds \$1000?"

Inputs: start = 500, growth = 10%, goal = 1000

Output: number of years

Steps by hand (pseudocode)

1. balance = 500, years = 0.
2. While balance is not above 1000: add 10% interest, add one to years.
3. Print years.

Python

```
balance = 500
years = 0
while balance <= 1000:
    balance = balance * 1.10
    years += 1
print(years)           # 8
```

TIPS AND TRAPS

- Use **for** when you know how many times or what to loop over. Use **while** when you wait for something to happen.
- Infinite loop? You forgot to update the variable in the condition.
- Check the condition direction: loop WHILE the goal is NOT yet reached.
- Validation pattern: ask once, **while** the answer is bad, ask again.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Print 1 to 5 using a while loop.
2. **COMBO** Keep asking for a number until the user enters one between 1 and 10.
3. **COMBO** Halve 1000 repeatedly until it is below 1. Print how many halvings it took.
4. **REAL** ATM PIN check: 3 attempts allowed. Stop on success. Print Locked after 3 failures.

break and continue

break leaves the loop entirely. continue skips the rest of this round and goes to the next one.

KNOW THIS

```
for x in nums:
    if x < 0:
        continue          # skip negatives, go to next x
    if x == 99:
        break              # stop everything
    print(x)

for x in nums:
    if x == target:
        print("found")
        break
else:
    # runs only if the loop did NOT break
    print("not found")
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Read numbers from the user until they type 0. Ignore negative numbers. Print the sum of the others."

Inputs: numbers typed one at a time

Output: the sum of the non-negative numbers

Steps by hand (pseudocode)

1. total = 0.
2. Repeat forever: read a number.
3. If it is 0, stop (break).
4. If negative, skip (continue).
5. Otherwise add it to total.
6. After the loop, print total.

Python

```
total = 0
while True:
    n = int(input("Number (0 to stop): "))
    if n == 0:
        break
    if n < 0:
        continue
    total += n
print(total)
```

TIPS AND TRAPS

- **break** and **continue** only affect the INNERMOST loop.
- **continue** in a **while** loop: make sure the update line is not skipped, or you loop forever.
- Searching? **break** as soon as you find it, no need to keep looking.
- Put the stop check first, so it can never be skipped.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Print 1 to 10 but skip 5.
2. **COMBO** Print numbers 1 to 100 and stop at the first multiple of 17.
3. **COMBO** Loop through a list of prices. Skip anything above 100. Print the rest.
4. **REAL** Process a list of transactions. Stop at the first one marked FRAUD and report its position.

Pattern: Sum (accumulator)

Start a variable at 0 BEFORE the loop. Add to it on every round. This shape solves 'total', 'sum', 'add up'.

KNOW THIS

```
total = 0                # 1. start BEFORE the loop
for x in nums:           # 2. visit each item
    total += x           # 3. accumulate
print(total)             # 4. use the result AFTER the loop

# same shape, different jobs:
revenue += price * qty   salary_total += salary
distance += leg          points += gained
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Given the steps walked each day of a week, print the weekly total."

Inputs: steps = [4200, 8100, 6500, 3000, 9900, 12000, 7000]

Output: total steps

Steps by hand (pseudocode)

1. total = 0.
2. For each day's steps: add them to total.
3. Print total after the loop.

Python

```
steps = [4200, 8100, 6500, 3000, 9900, 12000, 7000]
total = 0
for day_steps in steps:
    total += day_steps
print(total)          # 50700
```

TIPS AND TRAPS

- The most common bug: **total = 0** INSIDE the loop. It resets every round.
- **print(total)** inside the loop prints a running total, not the answer.
- Totals of floats: round at the END, **round(total, 2)**.
- Python has **sum(nums)**, but learn the loop first. You need it for harder problems.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Sum the numbers 1 to 100 with a loop.
2. **COMBO** Sum only the even numbers in a list.
3. **COMBO** Given prices and quantities as two lists, compute the total cost.
4. **REAL** A shopping cart is a list of (item, price) pairs. Print the subtotal, 8% tax, and total.

Pattern: Count

Same shape as sum, but you add 1 only when an item meets a rule. This solves 'how many'.

KNOW THIS

```
count = 0
for x in nums:
    if x % 2 == 0:           # the rule
        count += 1         # add ONE, not x
print(count)

# the rule can be anything:
if temp > 30:               if name.startswith("A"):           if score < 50:
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Given a week of daily temperatures, count how many days were above 30 degrees."

Inputs: temps = [28, 31, 35, 29, 33, 30, 27]

Output: the number of days above 30

Steps by hand (pseudocode)

1. count = 0.
2. For each temperature:
3. If it is above 30, add 1 to count.
4. Print count.

Python

```
temps = [28, 31, 35, 29, 33, 30, 27]
hot_days = 0
for t in temps:
    if t > 30:
        hot_days += 1
print(hot_days)      # 3
```

TIPS AND TRAPS

- Count adds 1. Sum adds the value. Do not confuse them.
- 30 itself is not counted because the rule says above (>). Check the boundary.
- Counting TWO different kinds? Use two counters.
- Percentage = `count / len(items) * 100`. Guard against an empty list.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Count how many numbers in a list are negative.
2. **MICRO** Count how many words in a list start with a capital letter.
3. **COMBO** Count odd numbers AND even numbers in one pass.
4. **REAL** Test scores: count how many passed (>= 50) and print the pass percentage.

Pattern: Max and Min

Remember the best value so far. Compare each new item to it. Replace it if the new one is better.

KNOW THIS

```
best = nums[0]           # start with the first item (list must not be empty)
for x in nums:
    if x > best:          # use < for minimum
        best = x
print(best)

# need WHERE it was? remember the index too:
best_i = 0
for i in range(len(nums)):
    if nums[i] > nums[best_i]:
        best_i = i
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Given test scores, print the highest score and which student (position) got it."

Inputs: scores = [72, 91, 65, 88]

Output: highest score and its position

Steps by hand (pseudocode)

1. Assume the first score is the best (position 0).
2. Look at each score by position.
3. If it beats the best, update best and its position.
4. Print both.

Python

```
scores = [72, 91, 65, 88]
best_i = 0
for i in range(len(scores)):
    if scores[i] > scores[best_i]:
        best_i = i
print(scores[best_i], best_i)    # 91 1
```

TIPS AND TRAPS

- Do NOT start **best** = 0. If all numbers are negative you get the wrong answer. Start from the first item.
- **max(nums)** and **min(nums)** exist, but interviewers want the loop.
- Ties: **>** keeps the FIRST maximum. **>=** keeps the LAST. Decide which is wanted.
- Second largest: track **best** and **second**, or sort the unique values.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Find the smallest number in a list.
2. **COMBO** Find the longest word in a list.
3. **COMBO** Find the largest and smallest in ONE loop.
4. **REAL** Sensor readings: report the highest reading, the lowest, and the gap between them.

Pattern: Average

Average = total / count. You already know how to sum and count. The new skill is guarding against an empty list.

KNOW THIS

```
total = 0
for x in nums:
    total += x
if len(nums) > 0:
    average = total / len(nums)
else:
    average = 0          # or None, or print a message
print(round(average, 2))

# conditional average: only items that qualify
total = 0
count = 0
for x in nums:
    if x >= 0:
        total += x
        count += 1
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Print the average of a student's grades, ignoring any grade of -1 (missing)."

Inputs: grades = [80, -1, 90, 70]

Output: average of valid grades, 2 decimals

Steps by hand (pseudocode)

1. total = 0, count = 0.
2. For each grade: if it is not -1, add it and count it.
3. If count is 0, there is no average. Otherwise total / count.
4. Print rounded.

Python

```
grades = [80, -1, 90, 70]
total = 0
count = 0
for g in grades:
    if g != -1:
        total += g
        count += 1
if count == 0:
    print("No grades")
else:
    print(round(total / count, 2))    # 80.0
```

TIPS AND TRAPS

- Dividing by zero crashes (**ZeroDivisionError**). Always ask: can the count be 0?
- Use the **COUNT** of qualifying items, not **len(nums)**, when you skip some.
- **/** always gives a decimal. **//** throws away the decimals.
- Keep **total** and **count** separate until the very end.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Average of 5 numbers typed by the user.
2. **COMBO** Average of only the positive numbers in a list.
3. **COMBO** Print every value that is above the average.
4. **REAL** Employee salaries: print the average and the number of people paid above average.

Pattern: Filter and Transform

Build a NEW list. Filter keeps items that pass a rule. Transform changes every item. Start with an empty list, append inside the loop.

KNOW THIS

```
cheap = []                                # 1. empty result list
for p in prices:
    if p < 10:                             # filter: only some items
        cheap.append(p)

with_tax = []
for p in prices:
    with_tax.append(p * 1.08) # transform: every item changed

# both together: keep cheap items AND discount them
sale = []
for p in prices:
    if p < 10:
        sale.append(p * 0.9)

# shortcut (learn the loop first):
cheap = [p for p in prices if p < 10]
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"From a list of prices, make a new list of the prices under \$10, each with 8% tax added."

Inputs: prices = [4.5, 12, 9.99, 25, 3]

Output: a new list of taxed cheap prices

Steps by hand (pseudocode)

1. result = [] (empty).
2. For each price: if it is under 10, add price * 1.08 to result.
3. Print result.

Python

```
prices = [4.5, 12, 9.99, 25, 3]
result = []
for p in prices:
    if p < 10:
        result.append(round(p * 1.08, 2))
print(result)          # [4.86, 10.79, 3.24]
```

TIPS AND TRAPS

- The empty list goes BEFORE the loop. Inside it would reset every round.
- **append** changes the list and returns **None**. Never write `lst = lst.append(x)`.
- Want to remove items from the original? Build a new list of the ones to KEEP.
- Never remove from a list while looping over it. You will skip items.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** From a list of numbers, build a list of just the even ones.
2. **MICRO** Build a list of the squares of 1 to 10.
3. **COMBO** From a list of words, build a list of those longer than 4 letters, in uppercase.
4. **REAL** Transactions list: build a list of refunds (negative amounts) shown as positive numbers.

Pattern: Search

Does something exist? Where is it? Use a flag (found = False), loop, set it True and break when you find it.

KNOW THIS

```
found = False
for x in ids:
    if x == target:
        found = True
        break          # no need to keep looking
print(found)

# where is it?
position = -1          # -1 means not found
for i in range(len(ids)):
    if ids[i] == target:
        position = i
        break

# shortcuts: target in ids      ids.index(target) (crashes if missing)
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Given a list of employee IDs, print the position of ID 4821, or Not found."

Inputs: ids = [1003, 4821, 7712], target = 4821

Output: position or Not found

Steps by hand (pseudocode)

1. position = -1.
2. For each index: if ids[i] equals the target, save i and stop.
3. If position is still -1 print Not found, else print it.

Python

```
ids = [1003, 4821, 7712]
target = 4821
position = -1
for i in range(len(ids)):
    if ids[i] == target:
        position = i
        break
if position == -1:
    print("Not found")
else:
    print(position)          # 1
```

TIPS AND TRAPS

- Never `print("Not found")` inside the loop's else branch. It prints for every non-match.
- Initialize the flag BEFORE the loop; change it only on success.
- `x in list` is the one-line version. It loops internally, so it is $O(n)$.
- Many matches wanted? Do not `break`. Collect them into a list instead.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Check whether the number 7 is in a list.
2. **COMBO** Print the index of the first negative number, or -1.
3. **COMBO** Find the first word longer than 6 letters.
4. **REAL** Given orders as (id, status) pairs, find order 1042 and print its status.

List basics

A list holds many values in order. Each has a position (index) that starts at 0. Negative indexes count from the end.

KNOW THIS

```
nums = [5, 3, 9, 1]      empty = []
len(nums)                # 4
nums[0]    nums[2]       # 5    9    first, third
nums[-1]   nums[-2]      # 1    9    last, second to last
nums[1] = 7              # change an item
nums.append(4)           # add to the end
nums.pop()               # remove and return the last item
nums.remove(9)           # remove first VALUE 9
9 in nums                 # True / False
if not nums: ...         # True when the list is empty
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Given a list of three quiz scores, add a fourth, change the first to 100, and print the last score and the number of scores."

Inputs: scores = [70, 85, 90]

Output: the last score, and how many scores

Steps by hand (pseudocode)

1. Append the new score.
2. Set scores[0] to 100.
3. Print scores[-1] and len(scores).

Python

```
scores = [70, 85, 90]
scores.append(77)
scores[0] = 100
print(scores[-1], len(scores))    # 77 4
```

TIPS AND TRAPS

- Valid indexes are 0 to `len(nums) - 1`. `nums[len(nums)]` is an `IndexError`.
- `nums[-1]` is the safe way to get the last item. It fails on an empty list.
- `nums.remove(x)` raises an error if `x` is missing. Check `x in nums` first.
- `b = a` does NOT copy a list. Use `b = a.copy()`.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Make a list of 5 numbers. Print the first, last, and length.
2. **MICRO** Append 10 to a list, then remove the last item.
3. **COMBO** Replace every 0 in a list with -1.
4. **REAL** A waiting line: add 3 customers, serve (remove) the first, print who is next.

List methods and sorting

Methods change a list or ask it a question. Know which ones change the list and which return something.

KNOW THIS

```

nums.append(x)           # add at end                (returns None)
nums.insert(0, x)        # add at position
nums.extend([1, 2])      # add many
nums.pop()               # remove + return
nums.pop(i)              # remove by index
nums.remove(x)           # remove by value
nums.index(x)            # position of x (error if missing)
nums.count(x)            # how many x
nums.sort()              # sort IN PLACE, ascending   (returns None)
nums.sort(reverse=True)  # descending
sorted(nums)             # NEW sorted list, original untouched
nums.reverse()           # in place                  nums[::-1] # reversed copy
nums.copy()              # copy
nums.clear()             # empty list
people.sort(key=lambda p: p[1]) # sort pairs by 2nd field

```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Print the three highest scores from a list, highest first, without changing the original list."

Inputs: scores = [72, 91, 65, 88, 79]

Output: [91, 88, 79]

Steps by hand (pseudocode)

1. Make a sorted copy in descending order (sorted).
2. Take the first three with a slice.
3. Print.

Python

```

scores = [72, 91, 65, 88, 79]
top3 = sorted(scores, reverse=True)[:3]
print(top3)           # [91, 88, 79]
print(scores)         # unchanged

```

TIPS AND TRAPS

- `nums = nums.sort()` makes `nums` equal to `None`. `sort()` returns nothing.
- Need the original later? Use `sorted(nums)`, not `nums.sort()`.
- `pop()` is O(1) at the end but `pop(0)` and `insert(0, x)` are O(n).
- To sort by a field: `key=`. To sort descending: `reverse=True`.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Sort a list ascending, then descending.
2. **COMBO** Remove duplicates from a list but keep the first-seen order.
3. **COMBO** Find the second largest number in a list.
4. **REAL** Students as (name, score) pairs: print them ranked by score, highest first.

Looping with indexes

Three ways to loop a list. Pick by what you need: the value, the position, or both.

KNOW THIS

```
for x in nums:                # just the values
    print(x)
for i in range(len(nums)):    # positions: needed to CHANGE items
    nums[i] = nums[i] * 2
for i, x in enumerate(nums):  # position AND value
    print(i, x)

# compare neighbours: stop one early
for i in range(len(nums) - 1):
    if nums[i] > nums[i + 1]:
        print("not sorted at", i)

# two lists together
for name, score in zip(names, scores):
    print(name, score)
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Given a list of prices, apply a 10% discount in place to every price above 50."

Inputs: prices = [20, 80, 55, 10]

Output: the same list with big prices reduced

Steps by hand (pseudocode)

1. Loop over the POSITIONS (we need to modify items).
2. If prices[i] > 50, set prices[i] to prices[i] * 0.9.
3. Print the list.

Python

```
prices = [20, 80, 55, 10]
for i in range(len(prices)):
    if prices[i] > 50:
        prices[i] = prices[i] * 0.9
print(prices)          # [20, 72.0, 49.5, 10]
```

TIPS AND TRAPS

- Changing `x` in `for x in nums` does NOT change the list. Use the index.
- Off-by-one: `range(len(nums) + 1)` crashes with `IndexError`.
- Comparing `nums[i + 1]` requires stopping at `len(nums) - 1`.
- `enumerate(nums, start=1)` gives numbering that starts at 1.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Print each item with its position: 0: apple.
2. **COMBO** Double every item of a list in place.
3. **COMBO** Check whether a list is sorted ascending using neighbours.
4. **REAL** Given two lists names and scores, print the name of the person with the top score.

Slicing

A slice takes a piece of a list or string: [start:stop:step]. The stop is NOT included. Any part can be left out.

KNOW THIS

```
a = [10, 20, 30, 40, 50]
a[1:3]      # [20, 30]          index 1 and 2
a[:2]       # [10, 20]         first two
a[2:]       # [30, 40, 50]     from index 2 on
a[-2:]      # [40, 50]        last two
a[::2]      # [10, 30, 50]     every other
a[::-1]     # [50, 40, 30, 20, 10] reversed
a[:]        # a full copy
"hello"[1:4] # 'ell'           works on strings too
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

“Given a list of daily sales, print the first three days, the last two days, and the list reversed.”

Inputs: sales = [120, 90, 150, 80, 200]

Output: three printed lists

Steps by hand (pseudocode)

1. First three: slice from the start up to 3.
2. Last two: slice with -2 and no stop.
3. Reversed: step of -1.

Python

```
sales = [120, 90, 150, 80, 200]
print(sales[:3])      # [120, 90, 150]
print(sales[-2:])     # [80, 200]
print(sales[::-1])    # [200, 80, 150, 90, 120]
```

TIPS AND TRAPS

- Slices never crash on out-of-range numbers. They just return less. Indexes DO crash.
- `a[1:3]` has $3 - 1 = 2$ items. Length = stop - start.
- A slice makes a NEW list. Changing it does not change the original.
- Palindrome check: `s == s[::-1]`.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Print the middle three items of a 5-item list.
2. **MICRO** Print every third item of a list.
3. **COMBO** Print the first and last 2 characters of a string.
4. **REAL** Rotate a list left by 1: [1,2,3,4] -> [2,3,4,1].

Dictionary basics

A dictionary stores key -> value pairs. Use it for lookups: 'price OF this item', 'score FOR this name'.

KNOW THIS

```
prices = {"small": 10, "medium": 15, "large": 20}
empty = {}                                # empty dict (NOT set())
prices["large"]                           # 20      KeyError if missing
prices.get("huge")                        # None     safe
prices.get("huge", 0)                    # 0       safe with default
prices["xl"] = 25                         # add or  overwrite
del prices["xl"]                         # remove
"small" in prices                         # True    checks KEYS
len(prices)
for k in prices: ...                     # keys
for k, v in prices.items(): ...          # key and value
list(prices.values())
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"A cafe menu has prices. Given a drink name, print its price, or Not on menu."

Inputs: menu dict, drink (str)

Output: the price or Not on menu

Steps by hand (pseudocode)

1. Build the menu dictionary.
2. If the drink is a key in the menu: print menu[drink].
3. Otherwise print Not on menu.

Python

```
menu = {"latte": 4.5, "mocha": 5.0, "tea": 3.0}
drink = input("Drink: ")
if drink in menu:
    print(menu[drink])
else:
    print("Not on menu")
```

TIPS AND TRAPS

- Missing key with `[]` crashes (**KeyError**). With `.get(key, default)` it does not.
- `{}` is an empty DICT. An empty set is `set()`.
- Keys must be unchangeable (str, int, tuple). Lists cannot be keys.
- Keys are unique: assigning to an existing key overwrites it.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Create a dict of 3 countries and capitals. Print one capital.
2. **MICRO** Add a new country, then print how many entries there are.
3. **COMBO** Print every key and value on its own line.
4. **REAL** Inventory: given `{item: quantity}`, sell 2 of an item (only if enough stock), then print the stock.

Pattern: Frequency count

Count how many times each thing appears. The dictionary key is the thing, the value is its count. This is the single most reused interview pattern.

KNOW THIS

```
freq = {}
for x in items:
    freq[x] = freq.get(x, 0) + 1          # 0 if first time, then +1

# same thing, written out:
for x in items:
    if x in freq:
        freq[x] += 1
    else:
        freq[x] = 1

best = max(freq, key=freq.get)           # most common key
for k, v in freq.items():
    print(k, v)
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Count how many times each word appears in a sentence, and print the most common word."

Inputs: a sentence (str)

Output: counts per word, and the most common word

Steps by hand (pseudocode)

1. Split the sentence into words.
2. `freq = {}`.
3. For each word: `freq[word] = freq.get(word, 0) + 1`.
4. Find the key with the largest count and print it.

Python

```
sentence = "the cat and the hat and the bat"
freq = {}
for word in sentence.split():
    freq[word] = freq.get(word, 0) + 1
print(freq)                # {'the': 3, 'cat': 1, 'and': 2, ...}
print(max(freq, key=freq.get)) # the
```

TIPS AND TRAPS

- Memorize `freq[x] = freq.get(x, 0) + 1`. You will write it hundreds of times.
- Counting characters? Loop over the string instead of `split()`.
- Counting products, votes, orders, letters. Always the same shape.
- All counts equal 1 means no duplicates.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Count how many times each letter appears in a word.
2. **COMBO** Votes list: print the winner (most votes).
3. **COMBO** Check whether two words are anagrams using counts.
4. **REAL** Orders list of product names: print each product with its count, most popular first.

Lookup tables and grouping

Use a dictionary to replace a long if/elif chain, and to gather items into groups.

KNOW THIS

```
# lookup table: rules live in DATA, not in code
BASE = {"small": 10, "medium": 15, "large": 20}
price = BASE[size]                # BASE.get(size, 0) if size may be invalid

# group items by a key
groups = {}
for name, dept in staff:
    if dept not in groups:
        groups[dept] = []        # first time: make the list
    groups[dept].append(name)

# total by group
totals = {}
for cat, amount in expenses:
    totals[cat] = totals.get(cat, 0) + amount
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Expenses are (category, amount) pairs. Print the total spent per category."

Inputs: expenses = [("food", 12), ("rent", 500), ("food", 8)] **Output:** a total for each category

Steps by hand (pseudocode)

1. totals = {}.
2. For each (category, amount): add the amount to totals[category] (start at 0).
3. Loop the dict and print each category and total.

Python

```
expenses = [("food", 12), ("rent", 500), ("food", 8)]
totals = {}
for category, amount in expenses:
    totals[category] = totals.get(category, 0) + amount
for category, total in totals.items():
    print(category, total)        # food 20 / rent 500
```

TIPS AND TRAPS

- Unpack pairs in the loop header: `for category, amount in expenses:`.
- A dict of lists needs the list created before the first `append`.
- Lookup dicts make rules easy to change: edit the data, not the logic.
- Dict of dicts: `accounts["A1"]["balance"] += 50`.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Make a lookup dict for shipping rates by zone and print the rate for a zone.
2. **COMBO** Group names by their first letter.
3. **COMBO** Total the salary per department.
4. **REAL** Bank: accounts as {id: balance}. Apply a list of (id, amount) transactions. Skip unknown ids and overdrafts.

Sets and duplicates

A set stores unique values and answers 'have I seen this before?' almost instantly. Use it for duplicates, uniqueness and membership.

KNOW THIS

```
seen = set()                # empty set (NOT {})
s = {1, 2, 3}
s.add(4)                    # adding again does nothing
s.remove(4)                 # error if missing
s.discard(4)               # no error if missing
4 in s                      # fast membership test
set([1, 1, 2])              # {1, 2} removes duplicates
len(set(nums)) < len(nums) # True if the list has duplicates
a | b      a & b      a - b # union both only-in-a
# sets have no order and no indexes: s[0] is an error
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Given a list of transaction IDs, print each ID that appears more than once (each only once)."

Inputs: ids = [101, 205, 101, 330, 205, 101]

Output: [101, 205]

Steps by hand (pseudocode)

1. seen = set(), repeated = set().
2. For each id: if it is in seen, add it to repeated.
3. Otherwise add to seen.
4. Print repeated.

Python

```
ids = [101, 205, 101, 330, 205, 101]
seen = set()
repeated = set()
for x in ids:
    if x in seen:
        repeated.add(x)
    else:
        seen.add(x)
print(repeated)      # {101, 205}
```

TIPS AND TRAPS

- Need only 'is it present?' -> set. Need a count -> dict. Need order or indexes -> list.
- **in** on a list is O(n). On a set it is O(1). That is a big deal on large inputs.
- **list(set(nums))** removes duplicates but loses the order.
- Two Sum idea: remember seen values in a set or dict while you loop.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Print how many unique numbers a list contains.
2. **COMBO** Print True if a list contains any duplicate.
3. **COMBO** Given two lists of user IDs, print the IDs that are in both.
4. **REAL** Sign-up check: given existing emails and new emails, print the new ones that are already taken.

Tuples and unpacking

A tuple is a fixed group of values. It is how you return several things from a function and how you loop over pairs.

KNOW THIS

```
point = (3, 4)
x, y = point          # unpack into two variables
point[0]              # 3 (read only: point[0] = 9 fails)

def min_max(nums):
    return min(nums), max(nums) # returns a tuple
lo, hi = min_max([4, 9, 1])

pairs = [("Ana", 91), ("Bo", 78)]
for name, score in pairs:      # unpack in the loop header
    print(name, score)
best = max(pairs, key=lambda p: p[1]) # pair with highest score
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Write a function that returns both the cheapest and the most expensive price from a list."

Inputs: prices (list of numbers)

Output: two values: lowest, highest

Steps by hand (pseudocode)

1. Compute min and max (or loop for both).
2. Return them together.
3. The caller unpacks into two variables.

Python

```
def price_range(prices):
    return min(prices), max(prices)

low, high = price_range([4.5, 12, 9.99, 3])
print(low, high)          # 3 12
```

TIPS AND TRAPS

- Unpacking needs matching counts: 2 values -> 2 variables.
- Tuples can be dict keys (lists cannot): `grid[(row, col)]`.
- `a, b = b, a` swaps two variables using tuples.
- Need to change it later? Use a list. Fixed record? Tuple.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Swap two variables in one line.
2. **COMBO** Write `stats(nums)` returning min, max and average.
3. **COMBO** Loop over (name, age) pairs and print the oldest.
4. **REAL** Given (item, price, qty) rows, print each line total and the grand total.

Rules to code: add-ons

Many requirements are a base price plus optional extras. Base = a choice (if/elif or a dictionary). Extras = separate ifs.

KNOW THIS

```
# base: ONE of several -> if / elif / else (or a lookup dict)
# extras: ANY combination -> separate ifs, never elif

total = BASE[size]
if extra_cheese:
    total += 1
if bacon:
    total += 2 if size == "small" else 3    # extra depends on size

# quantity
total = total * quantity
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"A burger is \$8 (single), \$11 (double) or \$14 (triple). Add bacon: +\$2 on single, +\$3 otherwise. Add cheese: +\$1. Compute the total for a given order."

Inputs: size (str), bacon (bool), cheese (bool)

Output: total price (number)

Steps by hand (pseudocode)

1. Base price from size.
2. If bacon: add \$2 for single, \$3 for others.
3. If cheese: add \$1.
4. Return the total.

Python

```
def burger_price(size, bacon, cheese):
    base = {"single": 8, "double": 11, "triple": 14}
    total = base[size]
    if bacon:
        if size == "single":
            total += 2
        else:
            total += 3
    if cheese:
        total += 1
    return total

print(burger_price("double", True, True))    # 15
```

TIPS AND TRAPS

- Choice among options = **if/elif**. Optional extras = separate **ifs**.
- Write each rule as a comment first, then turn each comment into code.
- Test a case for each rule alone, then all rules together.
- Invalid input (**size** = "mega")? Decide what to do: error message, 0, or **.get()**.

NOW YOU TRY (close this page, blank file, no peeking)

1. **COMBO** Coffee: small 3, medium 4, large 5. Extra shot +1, oat milk +0.5. Compute the price.
2. **COMBO** Hotel: nights x rate, +\$15 breakfast per night, weekend nights cost 20% more.
3. **REAL** Phone plan: base by plan, +\$10 per extra line, +\$5 if international.
4. **REAL** Game scoring: 10 per coin, 50 per enemy, x2 multiplier if no damage taken.

Rules to code: tiers, caps, minimums

Most billing rules are tiers (different rates for different ranges), caps (never above) and minimums (never below). `min()` and `max()` make them one-liners.

KNOW THIS

```
total = min(total, 20)           # CAP: never above 20
total = max(total, 5)           # MINIMUM charge of 5

first = min(units, 100) * 0.10   # tier 1: first 100 units
rest = max(units - 100, 0) * 0.15 # tier 2: everything after
bill = first + rest

fee = 5 + 3 * (hours - 1)        # first hour 5, then 3 each extra
import math
hours = math.ceil(minutes / 60)  # started hours: 61 min -> 2

price = price * (1 - 20 / 100)   # 20% off
price = price * 1.08            # 8% tax
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"A parking garage charges \$5 for the first hour and \$3 for each additional hour. The daily maximum is \$20. Compute the charge for a given number of hours."

Inputs: hours (whole number ≥ 0)

Output: charge in dollars

Steps by hand (pseudocode)

1. 0 hours $\rightarrow$ 0.
2. Otherwise: $5 + 3 * (\text{hours} - 1)$.
3. Cap with $\min(\text{fee}, 20)$.
4. Return.

Python

```
def parking_fee(hours):
    if hours <= 0:
        return 0
    fee = 5 + 3 * (hours - 1)
    return min(fee, 20)

print(parking_fee(1), parking_fee(4), parking_fee(9))    # 5 14 20
```

TIPS AND TRAPS

- Check the exact boundaries: 0, 1, 2, the hour where the cap starts, far past the cap.
- 'Each additional hour' means subtract the first one before multiplying.
- Order matters: discount THEN tax, or tax THEN discount? Read the requirement.
- 'Partial hours count as full' $\rightarrow$ `math.ceil`.

NOW YOU TRY (close this page, blank file, no peeking)

1. **COMBO** Electricity: first 100 units \$0.10, next 100 \$0.15, above that \$0.20. Compute the bill.
2. **COMBO** Taxi: \$3 base + \$1.20 per km, minimum fare \$8.
3. **REAL** Shipping: free over \$50, otherwise by weight: up to 1 kg \$4, up to 5 kg \$8, above \$12.
4. **REAL** Premium 20% off, regular 10% off, orders under \$20 no discount, then add 8% tax.

Testing your code

Never trust code you have not run on cases where you know the answer. Test the boundaries, not just the typical case.

KNOW THIS

```
print(parking_fee(1))      # expect 5
print(parking_fee(10))    # expect 20

assert parking_fee(1) == 5      # silent if correct, crashes if wrong
assert parking_fee(10) == 20, "cap failed"

cases = [(0, 0), (1, 5), (2, 8), (6, 20), (50, 20)] # (input, expected)
for hours, expected in cases:
    got = parking_fee(hours)
    print("OK " if got == expected else "BAD", hours, expected, got)
```

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Test a function `is_valid_age(age)` that should accept 0 to 120 inclusive."

Inputs: age (int)

Output: True / False

Steps by hand (pseudocode)

1. List normal values.
2. List BOUNDARIES: -1, 0, 1, 119, 120, 121.
3. Add weird: huge number.
4. Write expected answers BEFORE running.

Python

```
def is_valid_age(age):
    return 0 <= age <= 120

cases = [(-1, False), (0, True), (1, True), (120, True), (121, False)]
for age, expected in cases:
    print(age, is_valid_age(age) == expected)
```

TIPS AND TRAPS

- Decide expected answers BEFORE you run. Otherwise you will believe the output.
- Five kinds: normal, boundary, empty/zero, single item, invalid.
- Float math: compare `round(x, 2) == 5.0`, not `x == 5.0`.
- A passing test only proves those cases. Look for what you did not test.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Write 5 test cases for a function that returns the maximum of a list.
2. **COMBO** Test a grade function around every grade boundary.
3. **COMBO** Write asserts for `average(nums)` including an empty list.
4. **REAL** Take one of your earlier solutions and try to break it with weird input.

Reading errors

Read the error from the BOTTOM line up. The last line tells you WHAT, the line number tells you WHERE.

KNOW THIS

```
Traceback (most recent call last):
  File "ex.py", line 9, in <module>      <- WHERE (line 9)
    print(nums[5])
IndexError: list index out of range      <- WHAT (read this first)
```

SyntaxError	missing : or) or quote, = instead of ==
IndentationError	uneven spaces
NameError	typo, or variable used before it exists
TypeError	wrong type: "a" + 5, forgot int(input())
ValueError	int("abc"), list.remove(missing)
IndexError	index past the end (off by one) or empty list
KeyError	dict key missing -> use .get()
ZeroDivisionError	dividing by 0 (empty average?)
AttributeError	method does not exist, or the variable is None

HOW A QUESTION IS SOLVED (the process you will copy every time)

"This crashes. Find the bug without running it: `nums = [1, 2, 3, 4]` then for `i in range(len(nums) + 1)`: `print(nums[i])`"

Inputs: a list of 4 numbers

Output: should print all four numbers

Steps by hand (pseudocode)

1. What are the valid indexes? 0, 1, 2, 3.
2. What does `range(len(nums) + 1)` produce? 0 to 4.
3. Index 4 does not exist -> `IndexError`.
4. Fix: remove the + 1.

Python

```
nums = [1, 2, 3, 4]
for i in range(len(nums)):      # was: len(nums) + 1
    print(nums[i])
```

TIPS AND TRAPS

- A `SyntaxError` may really be on the line ABOVE the one reported.
- No error but wrong answer? That is a LOGIC bug. Print values to find it.
- No output at all? You probably never called your function.
- `print(type(x))` solves most `TypeError`s in seconds.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** What error does `int("hello")` raise? Predict first, then run it.
2. **MICRO** Fix: `if x = 5:`
3. **COMBO** Why does `total = 0` INSIDE a loop give wrong sums?
4. **REAL** Take a function you wrote and deliberately cause each error type once, to recognize them.

Common logic bugs

Most wrong answers come from the same dozen mistakes. Learn their shapes so you can spot them on sight.

KNOW THIS

Off-by-one	<code>range(len(nums)+1)</code>	<code>range(1, n)</code> when you meant <code>n+1</code>
Wrong boundary	<code>></code> where <code>>=</code> was needed	("at least" vs "more than")
Wrong variable	added price instead of total	
Forgot return	function result is <code>None</code>	
Early return	return inside the loop on the first round	
Reset in loop	<code>total = 0</code> inside the for	
print vs return	prints but gives back <code>None</code>	
Alias not copy	<code>b = a</code> changes <code>a</code> too -> <code>b = a.copy()</code>	
Strings immutable	<code>s.upper()</code> alone changes nothing	
Type confusion	<code>input()</code> is text: <code>"10" < "9"</code> is <code>True</code>	
Integer division	<code>//</code> where <code>/</code> was needed	
Mutating a list	removing while looping over it	

HOW A QUESTION IS SOLVED (the process you will copy every time)

"This returns the wrong answer for [3, 5, 8]. Find the bug: `def total(nums): for x in nums: t = 0; t += x; return t`"

Inputs: `nums = [3, 5, 8]`

Output: expected 16, actual 8

Steps by hand (pseudocode)

1. Trace by hand: round 1 `t=0` then `t=3`. Round 2 `t` is RESET to 0 then 5.
2. Spot the problem: initialization inside the loop.
3. Also return should be after the loop.

Python

```
def total(nums):
    t = 0                # before the loop
    for x in nums:
        t += x
    return t             # after the loop (outside the for)
```

TIPS AND TRAPS

- Debug method: smallest failing input -> predict each line -> print values -> first line where reality differs is the bug.
- Fix ONE thing at a time, then rerun.
- Read your code aloud as the computer would, not as you intended.
- Log every bug you make in `ERROR_LOG.md`. Patterns will show up fast.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** Why does `nums = nums.append(5)` break the list?
2. **COMBO** Why does `for x in nums: nums.remove(x)` skip items?
3. **COMBO** Why is `0.1 + 0.2 == 0.3` False? How do you compare floats safely?
4. **REAL** Write a buggy function on purpose, then trace it by hand to find the bug.

Big-O basics

Big-O describes how the work grows as the input size n grows. Interviewers always ask for time and space complexity.

KNOW THIS

$O(1)$	constant	index, dict get/set, set add/in, append, pop()
$O(\log n)$	log	halving each step (binary search)
$O(n)$	linear	one loop over the data, sum, max, "x in list"
$O(n \log n)$		sorting
$O(n^2)$	quadratic	a loop inside a loop over the same data

list: `x in lst` is $O(n)$ set / dict: `x in s` is $O(1)$ on average
 Space: extra memory you create. A new list/dict sized by the input = $O(n)$.
 A handful of variables = $O(1)$.

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Find whether a list has duplicates. Compare two solutions and state their complexity."

Inputs: nums (list of n numbers)

Output: True or False

Steps by hand (pseudocode)

1. Slow: compare every pair -> loop inside a loop -> $O(n^2)$ time, $O(1)$ space.
2. Fast: remember seen values in a set -> one loop -> $O(n)$ time, $O(n)$ space.
3. State the trade-off: extra memory buys speed.

Python

```
def has_duplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:      #  $O(1)$ 
            return True
        seen.add(x)        #  $O(1)$ 
    return False
# time  $O(n)$ , space  $O(n)$ 
```

TIPS AND TRAPS

- Count loops: one loop is usually $O(n)$. A loop in a loop is $O(n^2)$.
- Replacing `x in list` inside a loop with a set check turns $O(n^2)$ into $O(n)$.
- Drop constants: $3n$ and n are both $O(n)$.
- Always say BOTH time and space.

NOW YOU TRY (close this page, blank file, no peeking)

1. **MICRO** What is the time complexity of finding the max of a list?
2. **COMBO** What is the complexity of looking up a key in a dict? A value in a list?
3. **COMBO** State the time and space of your frequency-count solution.
4. **REAL** Take any earlier solution with a nested loop. Can you remove one loop with a set or dict?

Interview protocol

When the interviewer says 'write the code', never go silent. Follow the same script every time. Talk through it. A simple correct solution beats an unfinished clever one.

KNOW THIS

- 1 Repeat the problem in your own words. (20 s)
- 2 Ask: types? negative? empty? duplicates? invalid input? (30 s)
- 3 State INPUTS and OUTPUT. Give one example with numbers. (20 s)
- 4 Explain your plan in plain English. (30 s)
- 5 Write pseudocode as comments. (1 min)
- 6 Turn each comment into code. Talk while typing.
- 7 Trace by hand with the example. Test an edge case. (1-2 min)
- 8 State time and space complexity.
- 9 Mention an improvement or an alternative.

HOW A QUESTION IS SOLVED (the process you will copy every time)

"Interviewer: "Given a list of order totals, return the total of orders over \$20, with a 10% discount applied to those orders.""

Inputs: order_totals (list of numbers)

Output: one number

Steps by hand (pseudocode)

1. Say: I will loop through orders, pick the ones over 20, discount them, and add them up.
2. Ask: can the list be empty? (then return 0)
3. Comments first, then code.
4. Test: [], [10], [25, 30], [20].

Python

```
def discounted_total(order_totals):
    total = 0 # empty list -> 0
    for amount in order_totals:
        if amount > 20: # strictly over 20
            total += amount * 0.9
    return round(total, 2)

print(discounted_total([25, 30, 10, 20])) # 49.5
```

TIPS AND TRAPS

- Freeze? Say "let me do an example by hand". Then write your steps as numbered lines.
- Start with the brute-force solution. Improve it afterwards.
- Name things well. It shows clarity.
- Say what you are testing and why before you run.

NOW YOU TRY (close this page, blank file, no peeking)

1. **REAL** Pick any exercise and answer steps 1 to 4 out loud before writing code.
2. **REAL** Do a mock: write the blank-file template below from memory.
3. **REAL** Time yourself: requirement to working code in 10 minutes.
4. **REAL** Explain your solution to a rubber duck, including complexity.

The blank-file template

Type this skeleton from memory at the start of any problem. Fill in the blanks. It gives you a first move so you never stare at an empty file.

SKELETON

```
def solve(data):
    # 1. handle empty / edge case
    if not data:
        return 0
    # 2. set up variables (total, count, best, result list / dict / set)
    result = 0
    # 3. loop
    for item in data:
        # 4. decide
        if item > 0:
            result += item
    # 5. return
    return result

# 6. test: normal, boundary, empty
print(solve([1, -2, 3]))    # expect 4
print(solve([]))           # expect 0
```

WHAT EACH STEP MEANS

1. Edge case first: empty input, zero, one item. Return something sensible.
2. Variables: what do I need to remember while looping? total, count, best, a result list, a dict, a set.
3. Loop: visit every item (or while until a stop).
4. Decide: if/elif for the rule from the requirement.
5. Return (or print) exactly what the requirement says.
6. Test: normal, boundary, empty.

Syntax flash drill

Cover the right column. Answer from memory, out loud. Every miss goes into ERROR_LOG.md.

#	Question	Answer
1	Last element of a list nums?	<code>nums[-1]</code>
2	Index of the value 9?	<code>nums.index(9)</code>
3	Loop through the values?	<code>for x in nums:</code>
4	Loop through the indexes?	<code>for i in range(len(nums)):</code>
5	Index and value together?	<code>for i, x in enumerate(nums):</code>
6	Add an item to the end?	<code>nums.append(x)</code>
7	Remove the last item?	<code>nums.pop()</code>
8	Remove the first 5?	<code>nums.remove(5)</code>
9	Is 5 in the list?	<code>5 in nums</code>
10	Create an empty dict?	<code>d = {}</code>
11	Safe dict get, default 0?	<code>d.get(key, 0)</code>
12	Count frequencies?	<code>freq[x] = freq.get(x, 0) + 1</code>
13	Create an empty set?	<code>s = set()</code>
14	Add to a set?	<code>s.add(x)</code>
15	Sort ascending in place?	<code>nums.sort()</code>
16	Sorted descending copy?	<code>sorted(nums, reverse=True)</code>
17	Reverse a string?	<code>s[::-1]</code>
18	Define a function?	<code>def name(params):</code>
19	Give a value back?	<code>return value</code>
20	Swap two variables?	<code>a, b = b, a</code>

Syntax flash drill

Cover the right column. Answer from memory, out loud. Every miss goes into ERROR_LOG.md.

#	Question	Answer
21	Is n even?	<code>n % 2 == 0</code>
22	Text input to whole number?	<code>int(input(...))</code>
23	Print 2 decimals?	<code>print(f'{x:.2f}')</code>
24	Loop 1 to 10 inclusive?	<code>for i in range(1, 11):</code>
25	Loop 10 down to 1?	<code>for i in range(10, 0, -1):</code>
26	Largest / total / count?	<code>max(n) sum(n) len(n)</code>
27	Key in dict?	<code>key in d</code>
28	Loop dict key + value?	<code>for k, v in d.items():</code>
29	First 3 items?	<code>nums[:3]</code>
30	Every second item?	<code>nums[::2]</code>
31	Split a sentence into words?	<code>s.split()</code>
32	Join words with spaces?	<code>' '.join(words)</code>
33	Cap a total at 20?	<code>total = min(total, 20)</code>
34	17 divided by 5: whole and remainder?	<code>17 // 5 = 3, 17 % 5 = 2</code>
35	Remove duplicates?	<code>list(set(nums))</code>
36	Copy a list?	<code>nums.copy() or nums[:]</code>
37	Loop forever with an exit?	<code>while True: ... break</code>
38	Check a key exists, then use it?	<code>if k in d: d[k]</code>
39	Does the list have duplicates?	<code>len(set(n)) != len(n)</code>
40	Most common key in freq?	<code>max(freq, key=freq.get)</code>